

AI Usage Report

AI Tools Used

- **Antigravity (Google DeepMind)** — primary AI coding assistant used throughout the project
 - Powered by Claude Sonnet (Thinking) and Gemini models depending on task complexity
 - **Google Gemini API** — integrated directly into the service for AI-powered parking recommendations (`/api/recommend`)
-

Tasks Supported by AI

1. Project Structure Analysis

- Analyzed the existing Next.js App Router project layout before every modification
- Identified which files were responsible for routing, state management, UI, and API logic

2. Feature Removal & Navigation Cleanup

- Removed the "정기결제 (Subscription)" tab and all related pages, types, state, and API routes
- Renamed nav label "내 주차장 등록" → "동네 주차장 등록" across all files
- Removed unused Lucide icon imports (`Zap`, `CreditCard`, `ArrowRight`, `CheckCircle2`) that were left over after feature deletion

3. EV Charging Station Feature Removal

- Located and removed `evStations` prop, `EvStation` interface, and map marker rendering from `parking-map.tsx`
- Fixed the resulting TypeScript build error in `results-client.tsx` where `evStations` was still being passed as a prop

4. Supabase Integration

- Diagnosed that `@supabase/supabase-js` was installed locally in `node_modules` but missing from `package.json`, causing Vercel deployment to fail with `Module not found`
- Added the dependency to `package.json` and re-ran `npm install` to sync `package-lock.json`
- Designed and wrote the `app/lib/supabase.ts` singleton client using environment variables with safe fallbacks

5. Database Schema Design

- Designed the complete Supabase PostgreSQL schema for the project
- Wrote the full `supabase-schema.sql` including table creation, foreign keys, constraints, and Row Level Security (RLS) policies

6. API Route Implementation

- Created and rewrote Next.js App Router API routes:

- `GET / POST /api/posts` — community board posts
- `GET / DELETE /api/posts/[id]` — post detail with view count increment
- `GET / POST / DELETE /api/comments` — comments using `community_comments` table
- `GET / POST /api/community-parkings` — neighborhood parking registration
- `GET / PATCH / DELETE /api/community-parkings/[id]` — vote (like/dislike) with duplicate prevention

7. Community Page Full Rewrite

- Removed the two-tab structure (공유주차장 + 후기게시판)
- Rewrote `app/community/page.tsx` as a single bulletin board with:
 - Post list, detail view, write view state machine
 - Supabase-backed CRUD for posts and comments
 - Nickname-based session login stored in `localStorage`
 - Proper loading states, error alerts, and submit-disable during async operations

8. Register Page Targeted Edits

- Removed the post-registration success banner ("등록 완료! 커뮤니티에 소개 글이 자동 발행되었습니다")
- Removed the "커뮤니티에서 추천 & 후기 보기" button from registered parking cards

9. Build Verification

- Ran `npm run build` after every major change to confirm zero TypeScript errors and successful static/dynamic page generation

Example Prompts

1. "정기결제 탭 아예 삭제하고 탭에 '내 주차장 등록' 키워드를 '동네 주차장 등록'으로 바꿔줘"
2. "Vercel 배포 시 `Module not found: Can't resolve '@supabase/supabase-js'` 오류가 발생한다. 반드시 확인하고 수정하라."
3. "`community_parkings` 테이블이 없다는 오류가 발생한다. 내가 Supabase SQL Editor에 그대로 붙여넣을 수 있는 SQL 스크립트를 작성하라."
4. "커뮤니티 탭 안의 '공유주차장' 기능을 제거하고 '주차장 후기 게시판'만 남겨라. 커뮤니티 진입 시 바로 게시판이 보여야 한다."
5. "동네 주차장 등록 탭에 등록된 주차장 카드에 뜨는 '커뮤니티에서 추천 & 후기 보기' 버튼 없애기. 꼭 1, 2 사항만 바꾸고 다른 부분은 그대로 두기."
6. "게시하기 클릭 후 아무 동작도 하지 않는다. 원인을 분석하고 수정하라."

AI Outputs We Modified

AI Output	What We Changed
<code>supabase-schema.sql</code>	Changed table name from <code>comments</code> to <code>community_comments</code> to avoid collision with Supabase internal naming; adjusted foreign key targets accordingly

AI Output	What We Changed
app/api/posts/route.ts	Removed <code>parking_id</code> and <code>category</code> fields that referenced a deleted feature; simplified the SELECT query to only join <code>community_comments</code>
app/api/community-parkings/route.ts	Removed the auto-post creation block that inserted into <code>community_posts</code> on parking registration, which was causing cascade failures when the table did not yet exist
app/community/page.tsx	Stripped parking-tab state variables (<code>selectedParking</code> , <code>linkedPosts</code> , <code>parkings</code> , <code>activeTab</code> , etc.) that AI initially left as dead code after the tab removal
app/lib/types.ts	Replaced generic <code>Comment</code> type with <code>CommunityComment</code> mapped to the actual DB table; removed legacy <code>ParkingSubscription</code> and <code>PostComment</code> types

Core Files We Can Explain

File	Role
app/layout-wrapper.tsx	Defines the top navigation bar and footer. Contains <code>NAV_ITEMS</code> array that controls which pages appear in the menu.
app/community/page.tsx	Single-board community page. Manages three views (list / write / detail) as component state, fetches all data from <code>/api/posts</code> and <code>/api/comments</code> .
app/register/page.tsx	Neighborhood parking registration form. Calls <code>/api/geocode</code> for lat/lng then POSTs to <code>/api/community-parkings</code> .
app/lib/supabase.ts	Supabase client singleton. Reads <code>NEXT_PUBLIC_SUPABASE_URL</code> and <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code> from environment variables.
app/lib/types.ts	Shared TypeScript interfaces for all Supabase table rows: <code>CommunityParking</code> , <code>CommunityPost</code> , <code>CommunityComment</code> , <code>Review</code> , <code>FavoriteParking</code> .
app/api/posts/route.ts	<code>GET</code> returns all posts with comment count; <code>POST</code> inserts a new post. Both use the <code>community_posts</code> Supabase table.
app/api/comments/route.ts	<code>GET</code> fetches comments by <code>post_id</code> ; <code>POST</code> inserts; <code>DELETE</code> removes by <code>id</code> . Uses <code>community_comments</code> table.
app/api/community-parkings/route.ts	<code>GET</code> returns registered shared parkings (optionally filtered by <code>promoted=true</code>); <code>POST</code> inserts a new parking entry.
app/api/recommend/route.ts	Calls Google Gemini API to generate a natural-language parking recommendation based on the search results passed from the frontend.

File	Role
<code>app/results/parking-map.tsx</code>	Renders an interactive Leaflet map with destination marker, radius circle, and parking lot markers. Receives <code>parkingLots</code> array as props.
<code>supabase-schema.sql</code>	Complete DDL for all five tables (<code>community_parkings</code> , <code>community_posts</code> , <code>community_comments</code> , <code>reviews</code> , <code>votes</code>) with RLS policies. Run this in the Supabase SQL Editor.
<code>package.json</code>	Lists all runtime dependencies including <code>@supabase/supabase-js</code> (required for Vercel deployment).

What We Learned

- **Dependency declaration matters for deployment.** A package installed in `node_modules` locally does not get installed by Vercel unless it is listed in `package.json`. The `@supabase/supabase-js` error taught us that `npm install <pkg>` without `--save` can leave a gap between local and production environments.
- **Database tables must exist before any code calls them.** The `community_parkings` "schema cache" error happened because the Supabase client tried to query a table that had never been created. Writing and running the SQL schema first is a prerequisite—no amount of code fixes can substitute for a missing table.
- **Consistent naming across SQL and code prevents silent bugs.** When the SQL schema used `community_comments` but the API code queried `comments`, Supabase returned an error at runtime rather than at build time. Keeping table names identical between the `.sql` file, API routes, Supabase `select()` join strings, and TypeScript types is essential.
- **Next.js App Router `useSearchParams()` requires a `<Suspense>` boundary.** Using the hook directly in a client component that is statically pre-rendered causes a build-time bail. Wrapping the component in `<Suspense>` resolves this cleanly.
- **Targeted edits are safer than full rewrites.** When the user asked to remove only two UI elements from the register page, applying surgical `replace_file_content` calls to only those specific lines—rather than rewriting the whole file—avoided accidentally breaking the surrounding form logic.
- **Environment variables must be configured in Vercel separately.** Even if `.env` is present locally, it is excluded by `.gitignore` and never reaches the Vercel build environment. Every `NEXT_PUBLIC_*` variable must be added manually in the Vercel project dashboard under Settings → Environment Variables.